

Week 1- Fundamentals

Condensed Study Notes

Generated: 2026-05-25 00:19 UTC

Week 1- Fundamentals

Welcome to the first week of our study! This section lays the groundwork for what we'll be learning.

Our instructors for this course are Dr. F Adamu-Fika and Ms. OV Okpoko.

Learning Objectives

This section outlines what we aim to achieve as we delve into the world of algorithms.

We will begin by understanding what algorithms are and why they are so crucial in computing. An **algorithm** is essentially a step-by-step procedure or a set of rules for solving a problem or completing a task. Think of it like a recipe for a computer – it tells the computer exactly what to do, in what order, to achieve a specific outcome.

Next, we will explore different ways to categorize algorithms. One way is by their **algorithmic paradigm**. This refers to the general approach or strategy an algorithm uses to solve a problem. Examples include:

- **Divide-and-conquer:** This strategy breaks down a large problem into smaller, more manageable sub-problems, solves each sub-problem independently, and then combines their solutions to solve the original problem. Imagine breaking a large jigsaw puzzle into smaller sections, solving each section, and then assembling them.
- **Greedy algorithms:** These algorithms make the locally optimal choice at each step with the hope of finding a global optimum. It's like choosing the quickest route at each intersection on a road trip, hoping it leads to the fastest overall journey.

Another way to classify algorithms is by their **function or problem type**. This groups algorithms based on what kind of task they are designed to perform. Common examples include:

- **Sorting algorithms:** These are used to arrange items in a specific order, such as alphabetically or numerically.
- **Searching algorithms:** These are used to find a particular item within a collection of data.

Finally, we will lay the groundwork for analysing algorithms. This involves understanding how to measure their **efficiency** (how much time and memory they use) and **complexity** (how their resource usage scales as the input size grows).

Definition of an Algorithm

An algorithm is essentially a step-by-step procedure or a formula designed to solve a specific problem. Think of it like a recipe for baking a cake: it provides clear instructions to achieve a desired outcome.

Algorithms have several key characteristics:

- **Input:** An algorithm can accept zero or more pieces of information to work with. For example, a sorting algorithm needs a list of items to sort.
- **Output:** An algorithm must produce at least one result or outcome. The sorting algorithm, for instance, outputs the sorted list.
- **Definiteness:** Every step in an algorithm must be clear and unambiguous. There should be no confusion about what to do next, just like each step in a recipe is precise.
- **Effectiveness:** All the operations or steps within an algorithm must be basic enough that they can actually be carried out. They need to be feasible and executable.
- **Finiteness:** An algorithm must always terminate after a finite number of steps. It shouldn't go on forever; it needs to reach an end point. A recipe, for example, ends when the cake is baked.

Importance and application of algorithms

Algorithms are not just theoretical concepts; they are the engines behind many of the technologies we use every day. Their importance lies in their ability to enable efficient problem-solving across a vast array of domains.

Think of an algorithm like a recipe. Just as a recipe provides step-by-step instructions to bake a cake, an algorithm provides step-by-step instructions to solve a computational problem. Without these clear instructions, a computer wouldn't know what to do.

Here are some key areas where algorithms are applied:

- **Navigation:** When you use a GPS app to find the quickest route, you're relying on pathfinding algorithms. These algorithms calculate the best sequence of roads to take, considering factors like distance and traffic.
- **E-commerce:** Online shopping platforms use algorithms for search functions (to find products you're looking for) and recommendation systems (to suggest items you might like based on your past behavior).
- **Cybersecurity:** Algorithms are crucial for protecting information. Encryption algorithms scramble data so only authorized individuals can read it, while hashing algorithms create unique digital fingerprints for data integrity.

Algorithms are fundamental to computer science and software development, forming the core of how software is built and how computers operate. They are applied in diverse fields such as managing computer networks, powering artificial intelligence (AI), and organizing vast amounts of data in databases.

Classification by Function or Problem Type

Algorithms can also be categorized based on the specific type of problem they are designed to solve. This classification helps us understand the wide range of computational tasks algorithms can handle.

Here are some common types:

- **Searching Algorithms:** These algorithms are used to find a specific piece of data within a larger collection of data. Think of looking for a particular book on a crowded bookshelf. Examples include

Binary Search and Linear Search.

- **Sorting Algorithms:** These algorithms arrange data into a specific order, such as ascending or descending. Imagine organizing a deck of cards by suit and rank. Popular examples are Quick Sort and Merge Sort.
- **Graph Algorithms:** These are designed to solve problems that involve 'graph' structures. A graph is like a network of connected points (called nodes or vertices) and lines (called edges) representing relationships. Examples include Breadth-First Search (BFS), Depth-First Search (DFS), and Dijkstra's algorithm.
- **Optimisation Algorithms:** These algorithms aim to find the best possible solution when there are certain limitations or 'constraints'. It's like finding the most efficient route to deliver packages to multiple locations while minimizing travel time. The Knapsack problem is an example.
- **String Algorithms:** These algorithms deal with manipulating and processing sequences of characters, known as 'strings'. A common task is finding if a specific pattern appears within a larger text, like searching for a word in a document. The Knuth-Morris-Pratt (KMP) algorithm is an example used for pattern matching.
- **Combinatorial Algorithms:** These tackle problems that involve combinations, permutations, or subsets of items. This often means exploring many possible arrangements or selections. The Traveling Salesman Problem, where you want to find the shortest possible route that visits a set of cities and returns to the origin, is a classic example.
- **Numerical Algorithms:** These algorithms are used for performing calculations, often involving approximations or estimations. They are crucial for tasks like finding the roots of equations or approximating areas under curves. The Newton-Raphson method for finding roots is one such example.
- **Decision Problems:** These algorithms answer a specific question with a simple 'yes' or 'no' output. For instance, determining if a particular statement can be logically satisfied. The satisfiability problem is a well-known example.

Classification by Algorithmic Paradigm

Algorithmic paradigms are high-level strategies that guide how we approach and solve computational problems. They offer different frameworks for designing algorithms.

Here are some common algorithmic paradigms:

- **Brute Force:** This approach involves exhaustively searching through all possible solutions to find the correct answer. It's like trying every single key on a massive keyring to open a lock.
- **Example:** Checking all possible subsets in the subset-sum problem.
- **Divide-and-Conquer:** This strategy breaks down a problem into smaller, independent subproblems, solves each subproblem, and then combines their results to form the final solution. Think of it like assembling a large jigsaw puzzle by first sorting pieces by color and then assembling smaller sections before putting them all together.
- **Examples:** Merge Sort, Quick Sort (for sorting); Fibonacci sequence, Knapsack problem.

- **Decrease-and-Conquer:** This approach solves a problem by reducing it to a smaller instance of the same problem and then building up the solution. It's like learning to tie your shoelaces by practicing with just one loop, then two, and so on.
- Types of Decrease-and-Conquer:
 - **Decrease by a Constant:** The problem size is reduced by a fixed amount in each step. Example: Insertion Sort.
 - **Decrease by a Factor:** The problem size is reduced by a constant fraction in each step. Example: Binary Search.
 - **Variable Size Decrease:** The problem size reduction varies. Example: Euclidean algorithm for finding the Greatest Common Divisor (GCD).
- **Dynamic Programming:** This paradigm solves problems by breaking them into overlapping subproblems and storing the results of these subproblems to avoid recalculating them. It's similar to keeping a cheat sheet for a game where you record effective strategies you've already discovered.
 - Example: Storing results for Fibonacci sequence calculations.
- **Greedy Algorithms:** These algorithms make the locally optimal choice at each step, hoping that this choice will lead to an overall optimal solution. It's like choosing the most appealing option right now, assuming it's the best path forward.
 - Example: Huffman Coding for data compression.
 - Example: Dijkstra's Algorithm for shortest paths (with non-negative weights).
 - Example: Prim's Algorithm for Minimum Spanning Tree (MST).
- **Transform-and-Conquer:** This approach involves transforming the problem into a different, usually easier, form that can then be solved using other techniques. It's like changing the currency of a foreign country to your own to make transactions easier.
 - Types of Transform-and-Conquer:
 - **Representation Change:** Changing the way data is represented. Example: Balanced search trees.
 - **Instance Simplification:** Making the problem instance simpler. Example: Heapsort.
 - **Problem Reduction:** Transforming the problem into another known problem. Example: Transforming shortest path problems in graphs.
 - **Backtracking:** This is an incremental approach that builds solutions step-by-step. If at any point it determines that the current path cannot lead to a valid solution, it abandons that candidate and tries a different path. It's like exploring a maze and marking dead ends so you don't go down them again.
 - Examples: N-Queens problem, Sudoku solver.
 - **Approximation Algorithms:** These algorithms are used when finding an exact solution is computationally infeasible (too slow or requires too many resources). They find a solution that is close to the optimal one. This is useful for very complex problems like the Traveling Salesman Problem, where finding the absolute shortest route might take an impractical amount of time.

Structure of Algorithms

The structure of an algorithm refers to how its steps are organized, rather than what the algorithm actually does. Think of it like the blueprint of a building – it shows how the rooms are arranged, not what the

building is used for (e.g., a house, an office).

One common way to structure algorithms is using an **iterative structure**. This means using loops to repeat a set of steps until a specific condition is met. It's like following a recipe that says, "Stir the batter for 5 minutes" – you repeat the action of stirring until the time is up.

Another structural approach is when an algorithm **calls itself** to solve smaller versions of the same problem. This is known as a recursive structure. It's like a set of Russian nesting dolls, where each doll contains a smaller, identical doll inside, until you reach the smallest one.

Examples of Recursive structure

Recursion is a powerful way to structure algorithms where a function solves a problem by calling itself with smaller versions of the same problem. Think of it like a set of Russian nesting dolls: each doll contains a smaller, identical doll inside, until you reach the smallest one.

To understand how recursion works, let's look at a classic example: calculating the factorial of a non-negative integer.

Factorial Calculation:

The factorial of a non-negative integer 'n', denoted by $n!$, is the product of all positive integers less than or equal to n. For example, $5! = 5 \cdot 4 \cdot 3 \cdot 2 \cdot 1 = 120$.

Here's how we can define it recursively:

- **Base Case:** The simplest case, which stops the recursion. For factorial, the base case is when $n = 0$ or $n = 1$. In both cases, the factorial is 1.

Recursive Step: The part where the function calls itself with a smaller input. For factorial, if $n > 1$, then $n! = n \cdot (n-1)!$. This means to find the factorial of 'n', we multiply 'n' by the factorial of 'n-1'.

How it unfolds (e.g., for 3!):

1. `factorial(3)` is called.
2. Since 3 is not the base case (0 or 1), it goes to the recursive step: $3 * \text{factorial}(2)$.
3. `factorial(2)` is called. It's not the base case, so it becomes $2 * \text{factorial}(1)$.
4. `factorial(1)` is called. This is the base case! It returns 1.
5. Now, `factorial(2)` can finish: $2 * 1 = 2$.
6. Finally, `factorial(3)` can finish: $3 * 2 = 6$.

So, $3! = 6$.

Another common example is **Fibonacci Sequence generation**.

The Fibonacci sequence starts with 0 and 1, and each subsequent number is the sum of the two preceding ones (0, 1, 1, 2, 3, 5, 8, ...).

Recursively, this can be defined as:

- **Base Cases:** If $n = 0$, the result is 0. If $n = 1$, the result is 1.
- **Recursive Step:** If $n > 1$, the result is $\text{fibonacci}(n-1) + \text{fibonacci}(n-2)$.

This recursive structure mirrors the definition of the sequence directly, making it elegant. However, as we'll see later, this particular recursive approach can be inefficient because it recalculates the same values many times. This highlights an important aspect of algorithm design: while recursion can be clear, its efficiency needs careful consideration.

Algorithm EVENSUM

The EVENSUM algorithm is designed to calculate a specific sum.

Input: The algorithm takes one input, which is a positive integer denoted by 'n'.

Output: The algorithm's output is the sum of the nth even natural number. It's important to note that in this context, the first even natural number is considered to be 0. So, for $n=1$, the output is the sum of the 1st even number (0), for $n=2$, it's the sum of the 1st and 2nd even numbers ($0 + 2$), and so on.

Think of it like this: if you're asked to find the sum of the first 3 even numbers (starting from 0), you'd add $0 + 2 + 4$. The EVENSUM algorithm, given 'n', would compute this sum for you.

START

This section shows the continuation of the EVENSUM algorithm. It focuses on the iterative process of calculating the sum.

- The `while (i <= n)` loop condition means the algorithm will keep repeating the steps inside as long as the value of `i` is less than or equal to the input value `n`.
- Inside the loop, `even = even + (2i - 2)` updates the running sum. It adds the next even number in the sequence to the `even` variable. The expression `(2i - 2)` generates the sequence of even numbers (0, 2, 4, 6, ...).
- The `return EVENSUM n - 1 + (2n - 2)` line indicates the final calculation. It suggests that after the loop finishes, the final sum is computed by taking the sum calculated for $n-1$ and adding the n -th even number (represented here by $2n - 2$, although this notation seems slightly off if n is the count of even numbers). This hints at a recursive structure or a final step in an iterative one.

Examples of Iterative structure

This section continues our exploration of iterative algorithm structure, which uses loops to repeat a set of instructions. We'll look at a few more examples to solidify this concept.

Consider an algorithm that needs to calculate the sum of numbers from 1 to 100. An iterative approach would involve starting with a sum of 0 and then, in a loop, adding each number from 1 up to 100 to this running total. This is similar to how the EVENSUM algorithm worked, but instead of adding even numbers,

we'd be adding consecutive natural numbers.

Another common iterative task is finding the largest number in a list. You would initialize a variable to hold the 'largest found so far' (perhaps with the first number in the list). Then, you'd loop through the rest of the list. In each step of the loop, you compare the current number with the 'largest found so far'. If the current number is bigger, you update 'largest found so far' to be that current number. After checking every number, the variable will hold the absolute largest number in the entire list.

These examples illustrate how loops provide a structured way to repeat actions, making them powerful tools for solving problems that involve sequences or repeated operations.

Pros and Cons

When choosing between iterative and recursive algorithm structures, there are trade-offs to consider:

- **Recursive algorithms** tend to be easier to read and understand, often mirroring the problem's definition more directly. However, they can sometimes use more memory because each function call needs its own space on the call stack (think of it like stacking up many to-do lists, one for each step).
- **Iterative algorithms**, on the other hand, are generally more efficient when it comes to space. They typically use less memory because they don't create as many stacked function calls. Imagine a single, updated to-do list instead of many separate ones.

Efficiency and Complexity

Efficiency in algorithms refers to how quickly and effectively a problem is solved. It's about finding the best way to get the job done.

Complexity helps us understand and manage the resources an algorithm uses. This is especially important when dealing with large amounts of data, as it helps conserve valuable resources like processing time and memory.

There are two main aspects to consider when measuring an algorithm's complexity:

- **Time Complexity:** This measures how long an algorithm takes to run. We often think of this as the runtime, and it's usually expressed as a function of the size of the input.

Analogy:* Imagine you have to sort a deck of cards. Time complexity is like measuring how many shuffles and comparisons it takes to get them in order.

- **Space Complexity:** This measures how much memory an algorithm uses. It's about the storage space needed to run the algorithm.

Analogy:* Continuing with the card sorting example, space complexity would be like measuring how much table space you need to lay out the cards while you sort them.

Real-World Examples of Algorithms

Algorithms are not just theoretical concepts; they power many of the technologies we use every day. Here are some examples:

- **Pathfinding:** Algorithms like Dijkstra's Algorithm are used in GPS navigation systems to find the most efficient route between two points. Think of it like a smart map that calculates the quickest way to get you to your destination, considering traffic and road conditions.
- **Sorting:** Algorithms such as Quick Sort are crucial for organizing data. They are used in database indexing, which helps in quickly retrieving specific information from large collections of data.
- **Searching:** When you use a search engine, algorithms like Binary Search are at play. They efficiently locate specific information within a vast amount of data, much like quickly finding a word in a dictionary.
- **Cybersecurity:** Encryption algorithms are fundamental for protecting sensitive information. They scramble data so that only authorized individuals can read it, ensuring secure data transmission over networks.

Online resources

To further your understanding of algorithms, several online resources offer valuable supplementary material. These platforms provide different approaches to learning, from interactive visualizations to comprehensive course lectures.

Here are some recommended resources:

- **VisuAlgo:** This website offers visual representations of algorithms, which can be particularly helpful for understanding concepts like recursive and iterative structures through interactive animations. It's great for visual learners.
- **MIT OpenCourseWare – “Introduction to Algorithms”:** This free course from MIT provides lectures based on the renowned "Introduction to Algorithms" textbook. It's recommended for its coverage of algorithms, basic design paradigms, and complexity concepts.
- **GeeksforGeeks – Basics of Algorithms:** GeeksforGeeks features articles on introductory algorithm topics, including sorting, searching, and the details of recursive and iterative structures. You'll find sample code and explanations to aid your learning.
- **Coursera – “Algorithms Specialization” by Stanford University:** The first week of this specialization focuses on fundamental algorithmic concepts and the way we think about solving problems algorithmically, taught by Professor Tim Roughgarden.
- **Khan Academy – “Algorithms”:** Khan Academy provides beginner-friendly tutorials covering basic algorithm concepts, how algorithms are classified, and an introduction to complexity analysis. Interactive exercises are included to help solidify your understanding.